Groq LLM Feature Documentation

Feature: AI Operations Insight for admins.

Provider: Groq Chat Completions API.

Use Case

Admins can click `Generate Insight` on the analytics dashboard. The application gathers current stored metrics from the database, sends those metrics to Groq, receives an operations summary, and stores the generated response for later viewing on the dashboard.

Groq API Integration

The integration uses Groq's OpenAI-compatible chat completions endpoint:

```
POST https://api.groq.com/openai/v1/chat/completions
```

The API key is read from the environment through `GROQ_API_KEY`. The default model is `llama-3.3-70b-versatile`, configurable through `GROQ_MODEL`. Groq's official documentation lists the OpenAI-compatible base URL and chat completions API, and its model documentation lists `llama-3.3-70b-versatile`.

Configuration

Property	Environment Variable	Purpose
`groq.api-key`	`GROQ_API_KEY`	Secret token used in the `Authorization: Bearer` header.
`groq.model`	`GROQ_MODEL`	Model name sent in the Groq request. Default: `llama-3.3-70b-versatile`.
`groq.base-url`	`GROQ_BASE_URL`	Base URL for the OpenAI-compatible Groq API.

Code Structure

File	Responsibility
`GroqProperties.java`	Maps Groq configuration from `application.properties`.
`GroqInsightService.java`	Builds the prompt from stored metrics, calls Groq, extracts the response, and saves the insight.
`AiInsight.java`	JPA entity for persisted AI insight text, model name, title, and timestamp.
`AiInsightRepository.java`	Loads recent generated insights for display.
`ApiDashboardController.java`	Provides the admin POST route `/api-dashboard/insights/generate`.
`api-dashboard.html`	Displays the Generate Insight button and recent AI outputs.

Prompt Design

The prompt intentionally sends structured numeric metrics only. It includes API totals, failed requests, 24-hour health, active and cancelled service counts, endpoint usage, partner usage, and active service type summary. The system message instructs the model to act as a concise hospitality operations analyst and to use only supplied application metrics.

The user prompt asks Groq to return:

1. A three sentence executive summary.
2. Three specific observations.
3. Three recommended actions for the admin team.

Data Flow

1. Admin opens `/api-dashboard`.
2. Admin clicks `Generate Insight`.
3. Spring Security validates the admin session and CSRF token.
4. `ApiDashboardController` calls `GroqInsightService.generateOperationalInsight()`.
5. The service reads stored database metrics through JPA repositories.

6. The service sends a JSON request to Groq with model, temperature, max tokens, and messages.
7. The returned message content is saved into `ai_insights`.
8. The admin is redirected back to the dashboard where the new insight appears.

Error Handling

If `GROQ_API_KEY` is missing, the service throws a clear configuration error and the dashboard displays it as an admin-visible message. Runtime API errors are caught by the controller and returned as flash messages without breaking the dashboard page.

Security Notes

The Groq API key is never committed to source control. It is provided through environment variables. The LLM route is available only to admin users because it is under the existing admin-only dashboard flow.

Verification

The project test suite was run with `./gradlew.bat test` and completed successfully. The LLM call itself requires a real `GROQ_API_KEY` at runtime.